Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Операционные среды и системное программирование

ОТЧЁТ
к лабораторной работе №1
на тему:

УПРАВЛЕНИЕ ПРОЦЕССАМИ, ПОТОКАМИ И FIBER

Вариант 26

Выполнил Д.А. Скробот

Проверил Р.А. Шабанович

Минск 2026

СОДЕРЖАНИЕ

1 Номер и название варианта 3
2 Цель работы 4
3 Краткое описание алгоритма 5
4 Схема взаимодействия процессов и потоков 6
5 Перечень использованных функций API 8
6 Основные фрагменты исходного кода 9
7 Результаты выполнения программы 12
Заключение 14

1 НОМЕР И НАЗВАНИЕ ВАРИАНТА

Вариант 26: **Process против Thread**.

Требуется реализовать одну и ту же вычислительную задачу двумя способами:

– K дочерних процессов;

– K потоков одного процесса.

Используется одна и та же схема разбиения исходных данных. Исследуются значения $K = 1, 2, 4, 8$. Сравниваются время выполнения, стоимость создания исполнителей, используемые ресурсы и сложность получения общего результата.

2 ЦЕЛЬ РАБОТЫ

Изучить механизмы управления вычислительными процессами и потоками в операционной системе, получить практические навыки создания, завершения и ожидания процессов и потоков, измерения времени выполнения, обработки ошибок и исследования влияния параллельного выполнения на производительность системы.

В рамках работы необходимо:

– проверять корректность входных данных;

– обрабатывать ошибки с выводом названия операции, кода `errno` и текстового описания;

– корректно закрывать созданные дескрипторы;

– не оставлять запущенных процессов или потоков после завершения программы;

– выводить информацию о ходе работы;

– измерять время выполнения исследуемых операций;

– проверять корректность результата, сравнивая параллельный результат с последовательным.

3 КРАТКОЕ ОПИСАНИЕ АЛГОРИТМА

В качестве вычислительной задачи выбрано приближённое вычисление числа π методом Монте-Карло.

Для $N = 100\,000\,000$ точек генерируются случайные координаты (x, y) в единичном квадрате. Если $x^2 + y^2 \leq 1$, точка попадает в четверть единичного круга. Тогда

$$\pi \approx 4 \cdot \frac{\text{число попаданий}}{N}.$$

Исходные данные разбиваются на K частей:

$$n_i = \left\lfloor \frac{N}{K} \right\rfloor + \begin{cases} 1, & i < N \bmod K, \\ 0, & \text{иначе.} \end{cases}$$

Для каждого рабочего i используется детерминированный seed

$$seed_i = \texttt{0xC0FFEE} + i \cdot \texttt{0x9E3779B97F4A7C15}.$$

Это позволяет получить одинаковый результат для процессов и потоков при одном и том же K, а также вычислить эталонный результат последовательным суммированием.

Для процессов результат каждого дочернего процесса передаётся родителю через неименованный канал `pipe`. Для потоков результат записывается в общую структуру `ThreadArg`, после чего главный поток суммирует значения после `pthread_join`.

Время измеряется с помощью `clock_gettime(CLOCK_MONOTONIC)`. Каждый режим запускается не менее трёх раз, в отчёт заносится медиана.

4 СХЕМА ВЗАИМОДЕЙСТВИЯ ПРОЦЕССОВ И ПОТОКОВ

Взаимодействие процессов

Родительский процесс создаёт K неименованных каналов `pipe`. Для каждого канала создаётся дочерний процесс с помощью `fork`. В дочернем процессе закрывается ненужный конец канала (читающий), а также все ранее созданные каналы, чтобы не оставалось лишних открытых дескрипторов. Затем дочерний процесс вычисляет свою часть попаданий методом Монте-Карло, используя детерминированный seed, соответствующий его номеру. Полученное значение типа `uint64_t` записывается в свой канал через `write`. После этого дочерний процесс закрывает пишущий конец канала и завершается с кодом 0 через `_exit`.

Родительский последовательно читает из каждого канала дочерних процессов результат работы соответствующего дочернего процесса. После чтения из канала родитель закрывает читающий конец. Сумма всех прочитанных значений даёт общее число попаданий.

Далее родитель ожидает завершения каждого дочернего процесса с помощью `waitpid`. Для каждого проверяется, завершился ли он нормально и с каким кодом. Если был завершён сигналом, это также фиксируется. После успешного ожидания всех детей родитель освобождает память, выделенную под массивы дескрипторов и `pid`, и возвращает суммарный результат.

Таким образом, взаимодействие процессов включает:

- создание каналов `pipe` для передачи результатов;
- создание дочерних процессов через `fork`;
- передачу результата от ребёнка к родителю через `write`/`read`;
- ожидание завершения детей через `waitpid`;
- суммирование результатов в родительском процессе.

Взаимодействие потоков

В случае потоков главный поток создаёт K рабочих потоков с помощью `pthread_create`. Для каждого потока заранее подготавливается структура `ThreadArg`, содержащая номер потока, количество точек для обработки, seed и поле для результата. Указатель на эту структуру передаётся в функцию потока.

Все потоки находятся в одном адресном пространстве и имеют доступ к общим данным, поэтому нет необходимости в межпроцессном взаимодействии. Каждый поток вычисляет свою часть попаданий методом Монте-Карло и записывает результат в поле `hits` своей структуры `ThreadArg`. Никаких блокировок не требуется, так как каждый поток пишет только в свою структуру, а главный поток читает эти структуры только после завершения соответствующего потока.

Главный поток после создания всех рабочих потоков ожидает их завершения с помощью `pthread_join`. После успешного присоединения каждого потока главный поток суммирует значения `hits` из всех структур `ThreadArg`. Полученная сумма является общим числом попаданий.

Таким образом, взаимодействие потоков включает:

– создание рабочих потоков через `pthread_create`;
– передачу каждому потоку указателя на его структуру `ThreadArg`;
– запись результата каждым потоком в своё поле `hits`;
– ожидание завершения потоков через `pthread_join`;
– суммирование результатов в главном потоке.

Ключевое отличие от процессов состоит в том, что потоки используют общую память и не требуют каналов для обмена данными, что упрощает получение общего результата, но требует аккуратной работы с общими структурами данных.

5 ПЕРЕЧЕНЬ ИСПОЛЬЗОВАННЫХ ФУНКЦИЙ API

Так как работа выполнена на Linux/POSIX API, ниже приведены использованные функции Linux/POSIX и их аналоги в Windows API.

Таблица 5.1 – Использованные функции API

Linux/POSIX	Назначение	Аналог Windows API
`fork()`	создание дочернего процесса	`CreateProcess()`
`pipe()`	создание канала для IPC	`CreatePipe()`
`read()`, `write()`	обмен данными через канал	`ReadFile()`, `WriteFile()`
`waitpid()`	ожидание завершения процесса	`WaitForSingleObject()`, `GetExitCodeProcess()`
`kill()`	завершение процесса	`TerminateProcess()`
`pthread_create()`	создание потока	`CreateThread()`
`pthread_join()`	ожидание завершения потока	`WaitForSingleObject()`
`getrusage()`	информация о ресурсах	`GetProcessTimes()`
`syscall()`	получение TID потока	`GetCurrentThreadId()`

Дополнительно использовались `strerror()`, `calloc()`, `free()`, `strtoull()`, `printf()`, `fprintf()` и другие стандартные функции языка C/C++.

6 ОСНОВНЫЕ ФРАГМЕНТЫ ИСХОДНОГО КОДА

Измерение времени

```
static double now_seconds()
{
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0)
    {
        report_error("clock_gettime");
        return 0.0;
    }
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1e-9;
}
```

Вычисление попаданий методом Монте-Карло

```
static uint64_t monte_carlo_hits(uint64_t n, uint64_t seed)
{
    uint64_t state = seed;
    uint64_t hits = 0;
    for (uint64_t i = 0; i < n; ++i)
    {
        double x = uniform01(state);
        double y = uniform01(state);
        if (x * x + y * y <= 1.0)
            ++hits;
    }
    return hits;
}
```

Создание процессов и передача результата

```
if (pipe(pipes[i]) != 0)
{
    report_error("pipe");
    fatal = true;
    break;
}

pid_t pid = fork();
if (pid == 0)
{
    ::close(pipes[i][0]);
    //
    uint64_t my_n = N / K + ((uint64_t)i < (N % K) ? 1 : 0);
    uint64_t seed = make_seed(i);

    uint64_t hits = monte_carlo_hits(my_n, seed);
```

```
    ssize_t w = ::write(pipes[i][1], &hits, sizeof(hits));
    if (w != (ssize_t)sizeof(hits))
        _exit(2);

    ::close(pipes[i][1]);
    _exit(0);
}

pids[i] = pid;
::close(pipes[i][1]);
```

Создание потоков и сбор результатов

```
struct ThreadArg
{
    int idx;
    uint64_t n;
    uint64_t seed;
    uint64_t hits;
};

static void* thread_worker(void* arg)
{
    ThreadArg* a = (ThreadArg*)arg;
    a->hits = monte_carlo_hits(a->n, a->seed);
    return nullptr;
}

for (int i = 0; i < K; ++i)
{
    args[i].idx = i;
    args[i].n = N / K + ((uint64_t)i < (N % K) ? 1 : 0);
    args[i].seed = make_seed(i);
    args[i].hits = 0;

    int rc = pthread_create(&tids[i], nullptr, thread_worker, &args[i]);
    if (rc != 0)
    {
        errno = rc;
        report_error("pthread_create");
        fatal = true;
        break;
    }
}

for (int i = 0; i < created; ++i)
{
    pthread_join(tids[i], nullptr);
    total += args[i].hits;
}
```

Основной цикл эксперимента

```
const int Ks[] = {1, 2, 4, 8};
const int REPEATS = 3;

for (int ki = 0; ki < 4; ++ki)
{
    int K = Ks[ki];
    uint64_t ref_hits = compute_reference(N, K);

    for (int r = 0; r < REPEATS; ++r)
    {
        RunResult res_proc = run_processes(K, N);
        RunResult res_thr = run_threads(K, N);
        //      total_time, creation_time, work_time
    }
}
```

7 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ПРОГРАММЫ

Эксперименты проводились при $N = 100\,000\,000$. Каждый режим запускался 3 раза, в таблицах приведены медианы или средние значения.

Последовательный запуск

```
sequential: hits=78540182 pi=3.1416072800 time=0.220202
```

Сводная таблица по времени

Таблица 7.1 – Медиана общего времени и среднее время создания исполнителей

K	proc total, c	proc creation, c	thr total, c	thr creation, c
1	0.220908	0.000084	0.222636	0.000031
2	0.113243	0.000128	0.112194	0.000040
4	0.059936	0.000248	0.056771	0.000055
8	0.042017	0.000485	0.048974	0.000114

Таблица 7.2 – Среднее время непосредственно вычислений

K	proc work, c	thr work, c
1	0.221932	0.221602
2	0.113555	0.112218
4	0.060659	0.056476
8	0.041570	0.050445

Проверка корректности

Для всех K результаты процессов и потоков совпали с эталонным значением `compute_reference(N, K)`. Отличие оценки π от эталона равно $0.000 \cdot 10^{0}$. Например:

```
K=1 reference hits=78540182 pi=3.1416072800
K=2 reference hits=78531282 pi=3.1412512800
K=4 reference hits=78531096 pi=3.1412438400
K=8 reference hits=78530483 pi=3.1412193200
```

Для $K = 1$ результат совпал с последовательным запуском: `hits=78540182`, $\pi = 3.1416072800$.

Использованные ресурсы

```
self:     user=4.019 sys=0.004 maxrss=7344 minflt=557 majflt=1
          nvcsw=61 nivcsw=200
children: user=2.777 sys=0.002 maxrss=1508 minflt=1740 majflt=0
          nvcsw=38 nivcsw=180
```

Видно, что дочерние процессы имеют отдельное адресное пространство и собственный учёт ресурсов. Потоки же учитываются в статистике самого процесса, так как они разделяют его адресное пространство.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была реализована одна и та же задача вычисления числа π методом Монте-Карло для K дочерних процессов и K потоков одного процесса, где $K = 1, 2, 4, 8$.

Основные результаты:

– При $K = 1$ время процесса и потока близко к последовательному (0.220202 с): накладные расходы малы.

– При $K = 2$ время уменьшилось почти вдвое: процесс – 0.113243 с, поток – 0.112194 с (ускорение 1.95–1.98).

– При $K = 4$ процесс – 0.059936 с, поток – 0.056771 с (ускорение 3.7–3.9, уже меньше идеального).

– При $K = 8$ процесс – 0.042017 с, поток – 0.048974 с (ускорение 5.3 и 4.5; восьмикратного нет).

Рост числа исполнителей не даёт пропорционального ускорения по причинам:

1 Число ядер ограничено; избыточные исполнители выполняются по очереди.

2 При $K = 8$ растёт конкуренция за кэш, память и шину данных.

3 Создание, синхронизация, передача результатов и ожидание не распараллеливаются.

4 Создание процесса дороже потока (при $K = 8$: 0.000485 с против 0.000114 с), но на фоне общего времени это невелико.

5 Для процессов нужен IPC через `pipe` и `waitpid`; для потоков достаточно `ThreadArg` и `pthread_join`.

6 При $K = 8$ у потоков большой разброс (0.040870–0.061833 с) из-за планирования и миграции между ядрами, поэтому медиана хуже, чем у процессов.

Таким образом, процессы дают лучшую изоляцию, но требуют более сложного IPC и более дорогого создания. Потоки создаются быстрее и проще обмениваются результатами, но сильнее зависят от планировщика и конкуренции за общие ресурсы. В данном эксперименте наибольшее ускорение достигнуто при $K = 4$, а дальнейшее увеличение K до 8 даёт ограниченный выигрыш из-за аппаратных ограничений и накладных расходов.